

混合检索算法详解：从向量到

一、为什么需要混合检索？

在实际的应用中，纯向量检索存在一些固有的不足。向量检索擅长捕捉语义层面的相似性，比如「苹果很好吃」和「水果之王味道不错」在向量空间中距离很近。但当用户查询中包含专有名词、产品编号、日期、代码片段等精确信息时，向量检索的效果往往不如简单的关键词匹配。这是因为这些精确信息在训练语料中出现频率低，嵌入模型对它们的语义理解不够准确。

是一种经典的基于关键词的检索算法，源自的改进版。它在精确匹配场景下表现优异。如果你的知识库里有「产品型号」，用户也恰好搜了这个型号，那几乎一定能把它找出来。因此，将向量检索和结合起来进行混合检索，可以在语义理解和精确匹配之间取得很好的平衡。

二、余弦相似度原理

余弦相似度（）是向量检索中最常用的相似度度量方法。它的计算方式是：两个向量的点积除以它们各自模长的乘积，即。值域为，值越接近表示两个向量方向越一致，即语义越相似。与欧氏距离相比，余弦相似度对向量的绝对长度不敏感，更关注方向的一致性，这对于比较由不同长度文本生成的嵌入向量尤为重要。

在向量数据库中，查询过程本质上就是计算查询向量与所有文档向量之间的余弦相似度，然后返回相似度最高的一个。当数据量很大时，全量计算不可行，因此需要近似最近邻搜索（）算法，如、等通过索引结构来加速检索。算法在略微牺牲精度的条件下换取了数量级的搜索速度提升，使得百万级甚至十亿级向量的实时检索成为可能。

三、算法详解

全称是，是概率检索模型家族的标准版本。它的核心公式为：对于查询中的每个关键词，计算该词对文档的相关性得分，然后求和。单个词的得分公式为：乘以乘以除以加上乘以加上乘以除以。其中是逆文档频率，反映词的稀有程度；是词在文档中的出现频率；是文档长度；是所有文档的平均长度；和是可调节的参数。

公式中的参数含义如下。控制词频饱和度，默认值通常为到。越大，词频对得分的影响越线性；越小，词频增长带来的边际得分增量越小（即很快饱

和)。参数 `control` 控制文档长度归一化的程度，取值范围为 `0` 到 `1`，`0` 表示不做长度归一化，`1` 表示完全按文档长度惩罚。默认值 `0.75` 表示部分归一化。这两个参数使得比简单的 `tf-idf` 更灵活，能够适应不同场景的需求。

相比 `tf-idf` 的一个重要改进在于它考虑了词频饱和效应。在 `tf-idf` 中，如果一个词在文档中出现了 `10` 次，它的得分大约是出现 `1` 次时的 `10` 倍。但在实际的信息检索中，一个词出现 `10` 次已经足够说明它与文档相关了，出现 `100` 次并不会带来 `10` 倍的相关性提升。`BM25` 通过非线性函数对词频进行缩放，使得得分在词频增加到一定程度后趋于平缓，更符合实际检索场景。

四、混合检索的融合策略

混合检索的核心问题是如何融合向量检索和 `BM25` 的得分。最简单的方案是线性加权：最终得分等于 `bm25_score` 乘以向量得分加上 `vector_score` 乘以 `bm25_weight` 得分。`bm25_weight` 的取值范围为 `0` 到 `1`，可以根据实验结果调整。在实验中，`bm25_weight = 0.7` 等于 `0.7` 到 `0.9` 是一个不错的起点，但最优值因数据集和查询类型而异。

一个关键的技术细节是得分归一化。向量检索的得分和 `BM25` 的得分通常不在同一个量级上，直接加权会导致一方主导最终结果。常用的归一化方法包括 `l2` 归一化（将得分映射到 `[0, 1]` 区间）和 `z-score` 归一化（将得分转换为标准分数）。在实际代码中，通常先分别对两组得分做归一化处理，然后再进行加权融合。

另一种流行的融合策略是倒数排名融合（`Reciprocal Rank Fusion`，简称 `RRF`）。它的计算方式是：最终得分等于对每个检索列表求和，每一项为 `1 / (k + rank)` 加上 `1 / (k + rank)`，其中 `rank` 是文档在第 `k` 个检索列表中的排名，`k` 是一个常数（通常取 `60`）。`RRF` 的优点是它不依赖原始得分的数值大小，只看排名顺序，因此避免了得分归一化的问题。但缺点是它会丢失得分中包含的置信度信息，比如排名第 `1` 和第 `2` 的差距可能远比排名第 `10` 和第 `11` 的差距大。

五、实战建议

入门阶段建议先实现线性加权融合，因为它直观且容易调试。实现时注意三点：一是分别对两组得分做归一化；二是为向量检索和 `BM25` 各多召回一些候选（比如各取 `1000`），然后在融合阶段筛选出最终的 `top_k`；三是记录每次检索的中间结果（各来源得分、融合得分、最终排名），便于调试和优化。

在中文 `BM25` 场景下，`BM25` 还需要处理分词问题。英文天然以空格分隔单词，但中文需要先分词。推荐使用 `jieba` 分词库，它速度快、准确度也不错。核心代码就一行：

```
tokens = jieba.lcut(text)
```

。将分词后的结果作为 `BM25` 的输入即可。如果不做分词直接按字匹配（每个汉字作为一个 `token`），会导致检索效果大打折扣。